High-Performance Neural Decoding for Multiple-Access Channel Polar Codes: Overcoming Training Collapse and the Teacher-Forcing Performance Gap

The field of error-correction coding has undergone a transformative shift with the introduction of polar codes, the first class of codes theoretically proven to achieve the capacity of binary-input discrete memoryless channels under low-complexity successive cancellation decoding.[1] While their implementation in fifth-generation (5G) wireless standards for control channels has demonstrated their efficacy in single-user scenarios, extending polar coding to the Multiple-Access Channel (MAC) environment presents significant challenges in both analytical formulation and practical decoding.[1] Recent advancements in data-driven neural polar decoders (NPDs) have provided a framework for bypassing the rigid assumptions of classical channel models, utilizing the representative power of neural networks to learn optimal decoding functions directly from channel observations.[2] However, the transition from binary single-user decoding to 4-class joint multi-user decoding introduces a critical scalability bottleneck known as "training collapse" for large block lengths, alongside a persistent "performance gap" when using accelerated training techniques.[4] Addressing these issues requires a multi-faceted approach involving advanced training paradigms, symmetry-aware architectures, and information-theoretic curriculum design.

The Architecture of Neural MAC Polar Decoders and the Scalability Crisis

The foundational architecture of a neural polar decoder for a two-user Multiple-Access Channel is built upon the recursive structure of the Successive Cancellation (SC) algorithm.[4] In a standard polar decoder, the computation tree consists of nodes that perform (CheckNode) and g (BitNode) operations to update likelihood ratios or embeddings.[4] In the MAC setting, the decoder must resolve joint message bits (u, v) from two users, leading to a 4-class decision space at each leaf node: $\{(0,0),(0,1),(1,0),(1,1)\}$.[4] To handle the non-linearities and potential memory of the channel, the analytical 2×2 probability tensor operations—CalcLeft, CalcRight, and CalcParent—are replaced with weight-shared multilayer perceptrons (MLPs) operating on d-dimensional embeddings.[4]

While this neural successive cancellation (NSC) framework achieves near-optimal performance for short block lengths, such as $N = 32$ or $N = 64$, it suffers from a catastrophic training collapse as the block length N increases to 256 and beyond.[4] The root cause of this failure is the $O(N \log N)$ sequential depth of the decoding tree walk.[4] For a block length of $N = 256$, a single decoding pass involves approximately 1,500 sequential neural network operations.[4] During backpropagation through time (BPTT), the gradients must flow through this massive chain, leading to vanishing or exploding gradients that prevent the model from learning effective representations for the earlier stages of the decoding process.[4]

To quantify the performance of these decoders at manageable block lengths, empirical tests on the Gaussian Multiple-Access Channel (GMAC) reveal the effectiveness of neural architectures when training is successful.

Model Architecture	Parameters	Best BLER (N=32, 6dB)	Performance relative to SC	Training Depth
Analytical SC	-	0.046	1.0x	-
Sequential NSC ($d = 16$)	39K	0.046	1.0x	$O(N \log N)$
Sequential NSC ($d = 32$)	153K	0.037	0.8x	$O(N \log N)$
Fast_CE NSC ($d = 16$)	26K	0.340	7.4x	$O(\log N)$
WHT NSC (Large)	283K	0.254	5.5x	$O(\log N)$

The data demonstrates that while sequential training can match or even outperform analytical SC decoders, its $O(N \log N)$ depth makes it fundamentally unscalable for larger N.[4] Conversely, the "Fast_CE" technique, which utilizes teacher forcing to reduce the sequential depth to $O(\log N)$, enables training for any N but suffers from a significant performance gap, resulting in error rates roughly 5 to 8 times higher than the analytical baseline.[4]

The Fast_CE Paradigm and the Teacher-Forcing Paradox

The Fast Cross-Entropy (Fast_CE) technique is designed to parallelize the training of polar decoders across all positions at each tree depth.[4] In a single-user Neural Polar Decoder (NPD), this is achieved by providing the true bit values during the BitNode (g-node) operation.[4] Because the true bit u is known at training time, every node at a given depth d can be computed independently of the other nodes at the same depth, provided the embeddings from the layer above are available.[4] This parallelization reduces the sequential gradient depth from $O(N \log N)$ to $O(\log N)$, allowing the optimization of models for $N = 1024$ or higher.[4]

In the single-user case, the BitNode operation employs a residual connection of the form $output = MLP(e_{odd} \cdot u_{sign}, e_{even}) + e_{odd} \cdot u_{sign} + e_{even}$, where $u_{sign} = (-1)^u$.[4] This architecture possesses an inherent "sign-flip" symmetry: if the decoder makes an incorrect bit decision during inference, the resulting embedding is simply a mirror image of the "correct" embedding.[4] Since the training data consists of random bits, the model is exposed to both $+1$ and -1 sign patterns equally, making it robust to prediction errors.[4]

However, this robustness does not naturally extend to the 4-class MAC joint decoder. In the MAC setting, the joint bit-decisions (u, v) create a more complex conditioning space.[4] When the model makes an error during sequential decoding (inference), the BitNode receives one of three alternative conditioning patterns that it may never have encountered during teacher-forced training.[4] For example, if the true joint class is $(0, 0)$ but the model predicts $(1, 0)$, the embedding is corrupted in a way that is partially correct and partially wrong.[4] This leads to an "out-of-distribution" (OOD) problem where the model's internal states during inference differ significantly from those seen during training, causing errors to propagate and accumulate through the tree.[4]

Walsh-Hadamard Transform Decomposition and Spectral Insights

One sophisticated attempt to bridge the performance gap in 4-class MAC decoders involves the use of the Walsh-Hadamard Transform (WHT) to diagonalize the circular convolution operations.[4] In a two-user MAC, the CalcLeft operation (CheckNode) involves a coupling of four probability elements defined over the group $Z_2 \times Z_2$.[4] By transforming these probabilities into the WHT domain, the coupling is resolved into four independent scalar channels, where the CheckNode operation simplifies to element-wise multiplication.[4]

The 4 characters of the $Z_2 \times Z_2$ group provide specific sign-flip patterns for the BitNode:

1. Channel 0: Independent of (u, v); no sign flip occurs.
2. Channel 1: Sign flip dependent on user 2 bit v.
3. Channel 2: Sign flip dependent on user 1 bit u.
4. Channel 3: Sign flip dependent on the XOR sum $u \oplus v$.

Experimental results for WHT-based decoders at $N = 32$ reveal the impact of this decomposition on decoding reliability.[4]

WHT Model Variant	Sign Flip Logic	Params	Best BLER	vs. SC
WHT Shared Ops	Binary sign flip (single bit)	6K	0.606	13.2x
WHT Shared Ops	Character sign flips (all 4)	23K	0.336	7.3x
WHT Per-channel	Large MLPs	283K	0.254	5.5x
WHT + 10% Noise	Noisy teacher forcing	5K	0.378	8.2x

The transition from a simple binary sign flip to character-dependent sign flips significantly improved the BLER from 13.2x to 7.3x the SC baseline, confirming that the decoder must respect the group-theoretic structure of the MAC state space.[4] However, even with large, per-channel MLPs, the gap remained at 5.5x.[4] This plateau suggests that the bottleneck is not the mathematical coupling of the four tensor elements but rather the "exposure bias" inherent in the Fast_CE training objective.[4]

Analyzing Exposure Bias and Distribution Mismatch in Multi-Class Decoders

The "7x gap" observed across all Fast_CE variants—whether WHT-based or direct 4-class—is fundamentally a consequence of the train-test distribution mismatch in the BitNode.[4] In single-user binary decoding, an error flips the sign of the entire embedding, a transformation that preserves the "well-formedness" of the vector because the model is trained on a symmetric distribution of bits.[4] In the 4-class MAC case, an error in predicting (u, v) results in a "mixed" corruption of the WHT components.[4]

Consider a scenario where the true bits are $(0, 0)$, which corresponds to no sign flips across all four WHT channels.[4] If the model incorrectly predicts $(1, 0)$, the WHT channels 2 and 3 undergo a sign flip while channels 0 and 1 do not.[4] This specific pattern—partially flipped and partially original—is never seen during teacher-forced training where only "all-correct" sign patterns are presented.[4] Consequently, when the BitNode receives this corrupted input, its output becomes unpredictable, leading to a cascade of errors in subsequent tree levels.[4] Research into sequence-to-sequence models identifies this phenomenon as "exposure bias".[9] The model, having only ever seen ground-truth history during training, struggles to recover when it encounters its own imperfect predictions during inference.[9] For 4-class decoders, this is exacerbated by the fact that there are three distinct types of errors for every joint decision, each creating a unique out-of-distribution embedding pattern.[4] Simple "noisy teacher forcing," which adds random Gaussian noise to the labels during training, fails to resolve this gap because random noise does not accurately reflect the structured, correlated error patterns produced by the model's internal decoding logic.[4]

Parallel Scheduled Sampling as a Solution to Exposure Bias

To bridge the performance gap without sacrificing the $O(\log N)$ scalability of Fast_CE, advanced training techniques such as Parallel Scheduled Sampling (PSS) must be employed.[9] Scheduled Sampling aims to mitigate exposure bias by randomly replacing ground-truth tokens with the model's own predictions during training.[9] While traditional Scheduled Sampling is sequential and thus incompatible with the $O(\log N)$ training requirement, the PSS algorithm enables parallelization across time (or tree position).[12]

The PSS implementation for neural polar decoders involves a multi-pass approach.[13] In the first pass, the model performs a standard teacher-forced forward pass to generate its own predictions for every joint decision in the tree.[14] In the second pass, a mixture of true labels and model predictions is used to condition the BitNodes.[13] By setting the probability p of using model predictions, the model is gradually exposed to the "mixed" sign patterns it will encounter during inference.[4]

Empirical evidence from related sequence generation tasks shows that Parallel Scheduled Sampling can improve performance by up to 11.5% over teacher forcing while maintaining training speeds nearly identical to standard parallel methods.[12] For the 4-class MAC decoder, PSS allows the model to learn error-recovery mechanisms by seeing corrupted sign patterns during the gradient computation phase.[4]

Training Metric	**Sequential Scheduled Sampling**	**Parallel Scheduled Sampling (PSS)**	**Fast_CE (Teacher Forcing)**
Sequential Depth	$O(N \log N)$	$O(\log N) \times K$ passes	$O(\log N)$
Training Speed	Very slow	~25% slower than Fast_CE	Fastest
Error Robustness	High	High	Low (Exposure Bias)
Performance Gap	Minimal	Significantly Reduced	Large (~7x)

The integration of PSS into the neural MAC framework is predicted to be a key step in closing the 7x performance ceiling.[4] By using a modest number of passes (often $K = 1$ or $K = 2$), the model can effectively learn the mapping from corrupted inputs to correct outputs without reintroducing the sequential bottleneck.[13]

Symmetry and Equivariance in Neural Decoder Architectures

A second critical pathway for improving the robustness of multi-class decoders lies in the use of group-equivariant neural network architectures.[15] Group equivariance ensures that the symmetries of the input space are preserved through the layers of the network, preventing the model from having to "re-learn" the same transformation for different symmetric versions of the input.[16]
In the case of the 4-class MAC, the underlying symmetry group is the Klein four-group $V_4 \cong Z_2 \times Z_2$.[4] An equivariant architecture would ensure that if the input probabilities are permuted (e.g., by flipping the sign of user 1's bit), the output embeddings and subsequent decisions transform in a predictable, consistent manner.[17] This is particularly relevant for the BitNode, which must handle different sign-flip patterns.[4] By using layers that are equivariant to the V_4 group, the model can effectively generalize from one error pattern to all others.[15]

Architectural Principle	Implementation in MAC Decoder	Benefit
Group Equivariance	MLPs equivariant to $Z_2 \times Z_2$ permutations	Symmetric error handling [15]
Residual Connections	$e_{odd} \cdot signs +$ structure	"Well-formed" embeddings [4]
WHT-domain Processing	Independent channel processing	Decoupled feature learning [4]
Shared Weights	Universal f and g kernels across depths	Scaling to arbitrary N [4]

Research suggests that equivariant networks thrive in data-sparse settings and exhibit better calibration and confidence than standard MLPs.[15] For neural decoders, this means the model becomes less "confused" by the out-of-distribution patterns caused by inference-time errors, as its internal logic is mathematically constrained to respect the group-theoretic structure of the decisions.[15]

Curriculum Learning and Information-Theoretic Training Strategies

The training collapse at $N \geq 256$ is not only a consequence of gradient depth but also a result of the increasing complexity of the "reward" or loss landscape as the block length grows.[4] Curriculum learning (CL) offers a solution by presenting the model with increasingly difficult examples during the training process.[20] For polar codes, a principled curriculum can be

guided by the information-theoretic properties of the bit-channels.[21]
The CRISP (Curriculum based Sequential neural decoder) framework demonstrates that training a model on a nested hierarchy of subcodes can significantly improve reliability.[21] In a Polar(N, k) code, the decoder is first trained on lower-rate codes (small k) where the bit-channels are highly polarized and "easy" to decode, before moving to the higher-rate targets.[21] For the MAC scenario, this curriculum must be adapted to account for the multi-user capacity region.[3]
A 2-user MAC capacity region is defined by the constraints on the rates of the two users: $R_1 \leq C(X;Y|V)$, $R_2 \leq C(V;Y|X)$, and $R_1 + R_2 \leq C(X,V;Y)$.[7] The polarization of MAC channels occurs toward the vertices of this region.[7] A robust training strategy should involve:

1. Initial training on the most polarized bit-channels (those closest to the corners of the capacity region).[7]
2. Gradual introduction of "moderate" bit-channels that are less polarized and more prone to errors.[21]
3. Final fine-tuning on the full code structure at high SNR, where the model can learn fine-grained error correction.[4]

Curriculum Stage	Focus	Information-Theoretic Target
Initialization	Highly polarized bits	Near-zero or near-one bit-channel MI [7]
Expansion	Moderate reliability bits	Bridging the gap to the capacity boundary [22]
Refinement	Full block length $N = 256$	Minimizing BLER for the collective code [21]

This approach allows the model to stabilize its weights on "easy" decoding tasks before being subjected to the high-depth gradients of large N blocks.[21] Furthermore, combining CL with "DeepPolar" techniques—where the kernels themselves are non-linear and learned—can result in codes that exceed the performance of traditional linear polar codes in the short-to-medium block length regime.[23]

Hybrid Training: Fast_CE Pre-training and Sequential Fine-Tuning

One of the most promising strategies for solving the training collapse while closing the performance gap is a two-phase hybrid training paradigm.[4] This method acknowledges that while Fast_CE is efficient for initial weight discovery, the sequential tree walk is necessary for capturing the true "online" distribution of inference-time decoding.[4]

In Phase 1, the model is trained using the $O(\log N)$ depth Fast_CE objective.[4] This phase is computationally fast and allows the MLPs for CalcLeft and CalcRight to converge to a point where the decoder achieves a baseline level of performance (e.g., the 7.4x gap).[4] In Phase 2, the model switches to the $O(N \log N)$ sequential tree walk objective for a limited number of "fine-tuning" iterations.[4]

The advantage of this hybrid approach is that the model starts Phase 2 from a highly favorable initialization point rather than random weights.[4] Because the kernels are already functional, the gradients from the sequential pass are more meaningful and less prone to the "vanishing" problem seen in the $N = 256$ collapse.[4] Preliminary attempts at this were hindered by architectural mismatches between parallel and sequential models, but a unified architecture—using weight-shared MLPs and consistent embedding dimensions—enables seamless transition between the two training modes.[4]

Knowledge Distillation and Domain-Targeted Transfer Learning

For channels where an analytical SC decoder exists (such as the GMAC), knowledge distillation (KD) can be used to guide the neural decoder's training.[26] Knowledge distillation involves training a "student" model (the neural decoder) to mimic the "teacher" model (the analytical SC or a high-complexity SCL decoder).[27]

In the context of polar decoding, KD can be implemented at multiple levels:

1. **Response-Based Distillation:** The student matches the final 4-class logits produced by the teacher.[27] By matching soft probabilities rather than hard labels, the student learns the "dark knowledge" of the channel—such as which joint classes are likely to be confused.[27]
2. **Feature-Based Distillation (Snapshot Training):** The student attempts to match the internal probability tensors of the analytical decoder at every node in the tree.[4] While neural embeddings live in a different space than analytical tensors, a learned "projection head" can map the embeddings to the probability manifold.[4]
3. **Topology-Aware Distillation:** The student is regularized to preserve the structural relationships between nodes in the decoding tree, ensuring that the hierarchy of polarization is maintained.[28]

KD is particularly effective for bridging the performance gap because the teacher model (the analytical SC) already possesses the robustness to prediction errors that the Fast_CE model lacks.[4] By distilling the SC decoder's behavior under various noise conditions, the neural decoder can learn to be robust without requiring the full computational cost of sequential training.[26]

Modern Neural Decoders for 5G and Beyond: Insights

from Deletion and DNA Channels

Recent research by Aharoni et al. (2024-2025) has extended the Neural Polar Decoder framework to highly complex environments, such as 5G OFDM systems and DNA data storage pipelines.[1] These studies provide critical insights into how NPDs can handle non-standard noise and variable-length patterns, which are conceptually similar to the "corrupted" sign patterns in the 4-class MAC BitNode.[4]

In DNA storage, polar codes must handle synchronization errors, such as insertions and deletions, where the trellis-based ML decoder has $O(N^4)$ complexity.[32] The NPD for deletion channels utilizes four neural networks to replicate the SC operations but adapts the architecture to handle variable-length sequences.[32] A key finding from these papers is the "A" parameter—a tunable computational budget that allows the user to scale the complexity of the neural operations independently of the block length.[32]

Channel Type	NPD Adaptation	Scaling Insight
Unknown (MINE)	Learned embeddings $E_\phi : Y \rightarrow$	Decouples channel estimation from decoding [5]
Deletion Channel	Variable-length input handling	Robustness to positional shifts [34]
DNA Storage	Multiple trace processing (traces $y_{1:K}$)	Aggregation of noisy observations [32]
5G eMBB/BCH	OFDM and single-carrier adaptation	Robustness across Diverse Delay Profiles [1]

For the MAC decoder, the lesson from DNA storage is that NPDs can be trained to handle "corrupted" inputs if the training process incorporates the structural noise of the channel.[32] In the MAC case, this means the training set must include the specific "mixed" sign patterns caused by bit-level errors.[4] This reinforces the necessity of Parallel Scheduled Sampling or iterative refinement techniques that expose the model to its own systematic errors.[4]

The Road to Scalable MAC Neural Decoders: Strategic Recommendations

To resolve the training collapse at $N = 256$ and the 7x performance gap for 4-class MAC decoders, a combined strategy integrating architectural bias, advanced parallel training, and hybrid fine-tuning is required.

First, the "Fast_CE" training should be augmented with Parallel Scheduled Sampling (PSS). By performing a two-pass forward walk where Pass 1 generates predictions and Pass 2 conditions the BitNodes on a mixture of true and predicted labels, the model is exposed to the mixed sign patterns that cause the current 7x performance ceiling.[4] This approach maintains $O(\log N)$ sequential depth while providing the error-recovery training that standard teacher forcing lacks.[4]

Second, the decoder architecture should transition toward group-equivariant MLPs. By enforcing $Z_2 \times Z_2$ symmetry in the neural kernels, the model's parameter space is reduced and its robustness to sign errors is mathematically guaranteed.[15] This ensures that any learned error-correction logic is automatically applied across all three types of joint-bit errors, significantly improving generalization.[15]

Third, for large block lengths where sequential training is impossible from scratch, a hybrid "Pre-train and Fine-tune" approach should be adopted.[4] Phase 1 uses Fast_CE for efficient kernel initialization, and Phase 2 uses a sequential tree walk with detached gradients to close the performance gap.[4] Detaching gradients every $\log N$ steps limits the backpropagation depth while still allowing the model to see the "online" distribution of sequential decoding.[4]

Finally, the use of information-theoretic curriculum learning, inspired by the CRISP and DeepPolar frameworks, will allow the model to learn the hierarchical structure of polarization.[21] By starting with highly polarized bit-channels and gradually introducing complexity, the model can navigate the challenging loss landscapes of large N blocks without succumbing to vanishing gradients.[21] These advancements, supported by insights from 5G and DNA storage research, position neural polar decoders as a powerful and scalable solution for the next generation of multi-user communication systems.[1]

Works cited

1. A Study of Neural Polar Decoders for Communication - arXiv, accessed April 5, 2026, https://arxiv.org/pdf/2510.03069
2. Past Research Summary - Henry Pfister, accessed April 5, 2026, http://pfister.ee.duke.edu/research.html
3. Code Rate Optimization via Neural Polar Decoders - IEEE ISIT 2024 || Athens, Greece || 7-12 July 2024, accessed April 5, 2026, https://cmsworkshops.com/ISIT2024/view_paper.php?PaperNum=1653
4. fast_ce_mac_research.pdf
5. Data-Driven Neural Polar Codes for Unknown Channels With ... - arXiv, accessed April 5, 2026, https://arxiv.org/abs/2309.03148
6. Data-Driven Polar Codes for Unknown Channels With and Without Memory - bgu ee, accessed April 5, 2026, https://www.ee.bgu.ac.il/~haimp/Slides_polar_codes.pdf
7. Ziv Aharoni - Google Scholar, accessed April 5, 2026, https://scholar.google.com/citations?user=0ou0638AAAAJ&hl=iw
8. Understanding the Transformer Architecture: From Embeddings to Autoregressive Decoding | by Komal Mandal | Medium, accessed April 5, 2026, https://medium.com/@komalmandal1907/understanding-the-transformer-archite

cture-from-embeddings-to-autoregressive-decoding-7d637c0629b1
9. What is Scheduled Sampling? | Activeloop Glossary, accessed April 5, 2026, https://www.activeloop.ai/resources/glossary/scheduled-sampling/
10. On Exposure Bias, Hallucination and Domain Shift in Neural Machine Translation, accessed April 5, 2026, https://www.researchgate.net/publication/342850586_On_Exposure_Bias_Hallucination_and_Domain_Shift_in_Neural_Machine_Translation
11. An Introduction to the Encoder-Decoder Transformer with an English to Pig Latin Translator | by Isaac Edelman | Toward Humanoids | Feb, 2026 | Medium, accessed April 5, 2026, https://medium.com/correll-lab/an-introduction-to-the-encoder-decoder-transformer-with-an-english-to-pig-latin-translator-3993e77541dd
12. Parallel Scheduled Sampling | Request PDF - ResearchGate, accessed April 5, 2026, https://www.researchgate.net/publication/333717260_Parallel_Scheduled_Sampling
13. parallel scheduled sampling - arXiv, accessed April 5, 2026, https://arxiv.org/pdf/1906.04331
14. Parallel Scheduled Sampling, accessed April 5, 2026, https://arxiv.org/abs/1906.04331
15. (PDF) On Uncertainty Calibration for Equivariant Functions - ResearchGate, accessed April 5, 2026, https://www.researchgate.net/publication/396924082_On_Uncertainty_Calibration_for_Equivariant_Functions
16. arXiv:2104.13478v2 [cs.LG] 2 May 2021, accessed April 5, 2026, https://arxiv.org/pdf/2104.13478
17. Machine learning with generalised symmetries - UvA-DARE (Digital Academic Repository) - Universiteit van Amsterdam, accessed April 5, 2026, https://pure.uva.nl/ws/files/232500449/Thesis.pdf
18. Manipulating 3D Molecules in a Fixed-Dimensional E(3)-Equivariant Latent Space - OpenReview, accessed April 5, 2026, https://openreview.net/pdf/1fbd24a3d54d79fe137102c2a4c59664137e598c.pdf
19. Nested Construction of Polar Codes via Transformers - arXiv, accessed April 5, 2026, https://arxiv.org/abs/2401.17188
20. [PDF] CRISP: Curriculum based Sequential Neural Decoders for Polar Code Family, accessed April 5, 2026, https://www.semanticscholar.org/paper/CRISP%3A-Curriculum-based-Sequential-Neural-Decoders-Hebbar-Nadkarni/dbed08b81669fad453acb4683bfde562a3fcbced
21. CRISP: Curriculum based Sequential neural decoders for Polar ..., accessed April 5, 2026, https://proceedings.mlr.press/v202/hebbar23a/hebbar23a.pdf
22. Polar Coding Without Alphabet Extension for Asymmetric Models | Request PDF, accessed April 5, 2026, https://www.researchgate.net/publication/260303116_Polar_Coding_Without_Alphabet_Extension_for_Asymmetric_Models

23. Information Theory Feb 2024 - arXiv.org, accessed April 5, 2026, https://www.arxiv.org/list/cs.IT/2024-02?skip=35&show=2000
24. [PDF] Feedback is Good, Active Feedback is Better: Block Attention Active Feedback Codes | Semantic Scholar, accessed April 5, 2026, https://www.semanticscholar.org/paper/Feedback-is-Good%2C-Active-Feedback-is-Better%3A-Block-Ozfatura-Shao/984e2a6ff7fa5281e2170e699243c3a75b4ee4ec
25. DeepPolar: Inventing Nonlinear Large-Kernel Polar Codes ... - arXiv, accessed April 5, 2026, https://arxiv.org/abs/2402.08864
26. A Comprehensive Survey on Knowledge Distillation - arXiv, accessed April 5, 2026, https://arxiv.org/html/2503.12067v1
27. Knowledge Distillation in Neural Networks: A Comprehensive Survey - ResearchGate, accessed April 5, 2026, https://www.researchgate.net/publication/394846514_Knowledge_Distillation_in_Neural_Networks_A_Comprehensive_Survey
28. Topology-Guided Knowledge Distillation for Efficient Point Cloud Processing - arXiv, accessed April 5, 2026, https://arxiv.org/pdf/2505.08101
29. Enhancing Time Series Anomaly Detection: A Knowledge Distillation Approach with Image Transformation - MDPI, accessed April 5, 2026, https://www.mdpi.com/1424-8220/24/24/8169
30. A Lightweight Framework With Knowledge Distillation for Zero-Shot Mars Scene Classification | Request PDF - ResearchGate, accessed April 5, 2026, https://www.researchgate.net/publication/384468526_A_Lightweight_Framework_with_Knowledge_Distillation_for_Zero-Shot_Mars_Scene_Classification
31. Vision-Language-Vision Auto-Encoder: Scalable Knowledge Distillation from Diffusion Models - NeurIPS 2026, accessed April 5, 2026, https://neurips.cc/virtual/2025/poster/116899
32. Neural Polar Decoders for DNA Data Storage - arXiv, accessed April 5, 2026, https://arxiv.org/abs/2506.17076
33. A Study of Neural Polar Decoders for Communication - arXiv, accessed April 5, 2026, https://arxiv.org/abs/2510.03069
34. [2507.12329] Neural Polar Decoders for Deletion Channels - arXiv, accessed April 5, 2026, https://arxiv.org/abs/2507.12329